Open-World Terrain And Road Performance Patterns

Executive Summary

本仓库当前已经把地面道路从破碎的 3D 几何切换成了 page-based 贴地图层，但最新 profiling 仍显示整体帧时间约为 `28ms ~ 35ms/frame`，主热点已经转移到 `ground_mesh` 构建阶段，而不是道路 mask 本身。业界在这类问题上的主流做法非常一致：地形几何使用可复用的规则网格和高度场，而不是为每个 chunk 反复在 CPU 上生成独立三角形；普通地面道路则通常作为地形上的材质层、weight map、virtual texture 或 spline paint 处理，只有桥梁和高架保留真实 3D 几何 [1][2][3][4]。

对当前 Godot 项目而言，最现实的下一刀不是再继续微调 road mask，而是把 `ground_mesh` 从“chunk 内重复采样 + CPU 法线重建”的原型路径，升级为“共享规则网格 + 唯一顶点采样 + page 级 height/normal 缓存 + 主线程仅提交资源”的路径。这条方向同时符合经典 terrain rendering 论文、现代商用引擎文档，以及 Godot 自己对 `ArrayMesh`、线程和低层服务器的建议 [1][3][5][6][7][8][9]。

Key Findings

- **大世界地形的常见骨架不是任意 chunk 三角形重建，而是规则高度场 + 规则网格复用**：Geometry Clipmaps 使用嵌套规则网格并在视点移动时增量更新；Chunked LOD 则把地形预处理成可选块，在运行时只切换少量块 [1][2]。
- **唯一顶点复用是基本盘**：行业方案几乎都默认地形顶点来自规则网格，边界重复是有意且有限的，而不是在一个 chunk 里按三角形反复采样同一高度值 [1][2]。
- **地面道路通常与地形解耦**：Epic 的 `Landscape Splines` 可以直接改地形并把 blendmask 推进 terrain layer，`Runtime Virtual Texturing` 则把地形/道路等 shading 结果缓存到页化虚拟纹理中，低 mip 还能预烘焙或预流送 [3][4]。
- **后台准备、主线程提交是通用流式模式**：Godot 官方文档明确建议把重 CPU 工作放到线程/worker 中，但 scene tree 和渲染资源提交要走主线程或线程安全 API；需要大量对象时优先走 servers 级接口 [5][6][9]。
- **对 Godot 来说，`ArrayMesh` 比 `SurfaceTool` 更接近目标路径**：官方文档直接指出 `ArrayMesh` 更快，`SurfaceTool` 更像构建辅助器；如果每个 chunk 都在 mount 时重新 `generate_normals()`，很容易把 CPU 打穿 [7][8]。

Detailed Analysis

当前项目基线

本仓库本地 profiling 已经指向了非常清晰的边界：

- 运行时平均帧耗时仍在 `28.3ms ~ 34.6ms/frame`
- `update_streaming_avg_usec = 26142 ~ 32274`
- `streaming_mount_setup_avg_usec = 20051 ~ 25300`
- 单个 chunk setup 中 `ground_mesh_usec = 17528 ~ 18251`
- `ground_material_usec = 863 ~ 1526`
- 地形网格诊断显示：
  - `current_vertex_sample_count = 576`
  - `unique_vertex_sample_count = 169`
  - `duplicate_sample_count = 407`
  - `duplication_ratio = 3.408...`
  - `shaped_current_usec = 7513`
  - `shaped_unique_usec = 1637`

这说明当前第一矛盾不是“道路怎么画”，而是“地形网格如何生成”。外部资料也基本都支持这个判断。

1. 业界如何处理大地形网格

Geometry Clipmaps 的核心思想是：地形不是按 chunk 现做一堆独立 mesh，而是围绕观察者维护若干层嵌套的规则网格，每层都复用规则拓扑，只增量更新进入视野的新区域 [1]。Chunked LOD 的思想也类似，只是把世界离线切成可选块，在运行时选择少量块参与渲染，而不是重建全部几何 [2]。

这些方案虽然细节不同，但共同点非常明确：

- 网格拓扑高度规则化
- 顶点索引可复用
- 视点移动时尽量“滑动窗口”而不是“重新造网格”
- CPU 更像是在准备高度数据，而不是重建几何拓扑

这和当前项目里 `CityChunkScene._build_terrain_mesh()` 的路径有本质差异。当前路径更像原型验证：每个 chunk 独立构造地形三角形，并对共享顶点反复做昂贵的 `sample_height()`。如果 duplication ratio 已经到 `3.4x`，那就说明我们还没进入行业常见路径。

2. 业界如何处理高度采样与法线

规则高度场体系下，常见做法不是“按三角形逐点问高度”，而是：

- 先按规则分辨率采一张 height page / tile
- 每个唯一顶点只采一次
- 三角形索引直接复用
- 法线从高度差分推导，或者从缓存的 normal map/导数图中读出

Geometry Clipmaps 明确把高度和法线都当作可增量更新的数据页；GPU 版本甚至会在 GPU 上更新 normal map，而不是在 CPU 上对每块都重做完整几何法线 [1]。Godot 官方文档则指出 `ArrayMesh` 直接操作数组会比 `SurfaceTool` 更快，而 `SurfaceTool.generate_normals()` 需要完整 primitive 信息后再做一遍法线计算，这种便利 API 更适合原型或低频构建，不适合高频 mount 热路径 [7][8]。

对当前项目的直接含义是：

- 先把每个 terrain page 的唯一顶点高度数组缓存下来
- 使用共享 index buffer 渲染规则网格
- `sample_height()` 不再跟随三角形数量线性膨胀
- 法线优先改成“唯一顶点一次生成”或“从高度差分恢复”，不要在 chunk mount 时做整块 `generate_normals()`

3. 业界如何处理地面道路

现代开放世界里，“所有道路都做成真实 3D 路面实体”并不是常态。普通贴地道路通常会落到地形体系内部，典型做法包括：

- spline 驱动的 terrain deformation + layer paint
- splat/weight map
- decal/projected texture
- runtime virtual texture / page-based surface cache

Epic 的 `Landscape Splines` 官方文档明确支持两类关键动作：一类是用 spline 控制段编辑地形高度，另一类是把 blendmask 推到 terrain layer，从而让道路成为地形材质的一部分 [4]。`Runtime Virtual Texturing` 则提供了更“页化”的思路：把场景 shading 结果缓存到虚拟纹理页面里，低 mip 甚至可以预烘焙和预流送，而高 mip 只在需要时更新 [3]。

这和当前项目已经走出的 v4 路线其实是对的：普通道路用贴地的 page mask/overlay 表达，桥梁和高架再保留真实 mesh、厚度、碰撞和柱体。真正的问题不在“贴图路线是不是错了”，而在“地形网格本身还是太贵，拖垮了整个 mount 路径”。

4. 业界如何处理流式准备与主线程边界

Godot 官方关于线程和 servers 的建议很明确：

- `WorkerThreadPool` 适合并行 CPU 任务 [9]
- scene tree 不是线程安全的，资源提交要遵守线程安全边界 [6]
- 对于大量实例和底层渲染/物理对象，必要时用 servers 级 API 以避开 node 级开销 [5]

这对应到开放世界 terrain/road 流水线，常见模式通常是：

1. 后台线程准备 height page、road mask、normal page、placement 数据
2. 主线程只做 mesh/texture/resource commit
3. 可共享的数据页尽量跨 chunk/page 复用
4. 冷启动需要磁盘缓存或预烘焙，避免每次启动重算

我们 v4 在 road surface 这条线上已经开始这么做了，但 terrain 这条线还没完全跟上，所以热点自然转移到了 `ground_mesh`。

5. 对我们当前热点，业界通常怎么下刀

把外部资料与本仓库的 profiling 对上，最贴合行业常见处理方式的顺序是：

短期可落地方案

1. **把 terrain mesh 改成共享规则网格模板**
   - 预生成固定分辨率的 vertex/index 拓扑模板
   - 每个 chunk/page 只填唯一顶点高度与 UV
   - 直接把 `3.4x` 的重复采样先砍掉
2. **引入 page 级 height/normal 缓存**
   - 和 road surface page 对齐
   - 相邻 chunk 共享边界采样结果
   - 首次访问只为新进入页做增量更新
3. **把法线生成从 `SurfaceTool.generate_normals()` 热路径里拿掉**
   - CPU 侧：唯一顶点差分一次生成
   - 或 shader 侧：从高度图邻域差分恢复 normal
4. **保留“地面道路 = 地表覆盖层，高架 = 真几何”**
   - 不回退到全量 3D 路面
   - 让 road graph 继续做数据真相层

中期架构方案

1. **把 terrain 也页化到 shared page / clipmap 风格**
   - 不是 chunk 各造各的 mesh
   - 而是 page 或 ring 级别复用
2. **逐步向 clipmap / quadtree LOD 迁移**
   - 近景高分辨率
   - 远景只维护更粗的规则网格层
3. **把地面材质统一为 page atlas / virtual-texture-like 思路**
   - 道路
   - 条纹
   - 地表 roughness/albedo/normal
   - 都走同一套页缓存
4. **如果 GDScript 仍压不住首访成本，再考虑 GDExtension/C++**
   - 不是为“功能”上 C++
   - 而是只把最热的 height sampling / page build / mesh array fill 下沉

6. Godot 项目里最现实的下一刀

如果只从“最快见效”和“风险最低”来排序，当前最现实的下一刀是：

1. `CityChunkScene._build_terrain_mesh()` 退出现有的 triangle-by-triangle 采样路径
2. 引入一个共享的 `terrain grid template`（固定 resolution 的 vertices/indices）
3. 引入 `terrain page sample cache`
4. 将 `sample_height()` 限制为“每个唯一顶点只做一次”
5. 将法线从 `generate_normals()` 改为缓存或差分恢复

这一步做完之后，再去看是不是需要真正引入 clipmap/CDLOD 风格的更大改造。原因很简单：我们现在连“唯一顶点采样”这条地基都还没踩稳，直接跳到更复杂的 LOD 体系，工程风险会高于收益。

Areas of Consensus

- 大地形渲染的核心是**规则化**，不是任意 mesh 重建 [1][2]
- 背景准备、主线程提交是开放世界流式系统的常态 [3][5][6][9]
- 普通地面道路更适合作为地表材质层，而不是全量独立 3D 实体 [3][4]
- 共享 page / component / tile 是压低首次访问成本和移动成本的常见办法 [1][2][3]

Areas of Debate

- **Clipmaps vs Chunked LOD / CDLOD / 组件化地形**：本质都是规则高度场分层，只是数据结构、GPU/CPU 分工和 stitching 细节不同 [1][2]。
- **法线在 CPU 生成还是 shader 中恢复**：取决于目标平台、碰撞需求和材质复杂度；低模风格项目通常更偏向便宜的高度差分方案。
- **Virtual Texturing 是否值得完整引入**：大型商用引擎里收益很高，但在 Godot 项目里通常要先验证 page cache 的收益，再决定是否做更重的 VT 架构 [3]。
- **何时下沉到 C++/GDExtension**：如果 GDScript 已经被数据布局和算法改造压到合理水平，就未必需要；如果首访 page build 仍明显超线，再下沉热点更合理。

Sources

[1] Frank Losasso, Hugues Hoppe. *Geometry Clipmaps: Terrain Rendering Using Nested Regular Grids*. SIGGRAPH 2004. https://hhoppe.com/geomclipmap.pdf （同行评审论文，高可信）

[2] Thatcher Ulrich. *Chunked LOD*. Official project notes and materials by the original author, 2002. http://tulrich.com/geekstuff/chunklod.html （作者一手资料，高可信）

[3] Epic Games. *Runtime Virtual Texturing in Unreal Engine*. https://dev.epicgames.com/documentation/en-us/unreal-engine/runtime-virtual-texturing-in-unreal-engine （官方引擎文档，高可信）

[4] Epic Games. *Landscape Splines in Unreal Engine*. https://dev.epicgames.com/documentation/en-us/unreal-engine/landscape-splines-in-unreal-engine （官方引擎文档，高可信）

[5] Godot Engine. *Using Servers*. https://docs.godotengine.org/en/stable/tutorials/performance/using_servers.html （官方文档，高可信）

[6] Godot Engine. *Thread-safe APIs*. https://docs.godotengine.org/en/stable/tutorials/performance/thread_safe_apis.html （官方文档，高可信）

[7] Godot Engine. *ArrayMesh*. https://docs.godotengine.org/en/stable/classes/class_arraymesh.html （官方文档，高可信）

[8] Godot Engine. *SurfaceTool*. https://docs.godotengine.org/en/stable/classes/class_surfacetool.html （官方文档，高可信）

[9] Godot Engine. *WorkerThreadPool*.
https://docs.godotengine.org/en/stable/classes/class_workerthreadpool.html （官方文档，高可信）

Gaps and Further Research

- 还没有把 `CityChunkGroundSampler.sample_height()` 的内部成本继续细分到噪声层级、道路/地形融合层级和缓存命中层级。
- 还没有对“唯一顶点采样 + CPU 差分法线”和“shader 高度差分法线”做项目内 A/B profiling。
- 如果后续要把世界扩展到更大尺度，还需要专项研究 collision heightfield、navigation 和 minimap 数据流的 page 复用策略。